

U8: PL/SQL

Procedural Language extensions to the Structured Query Language

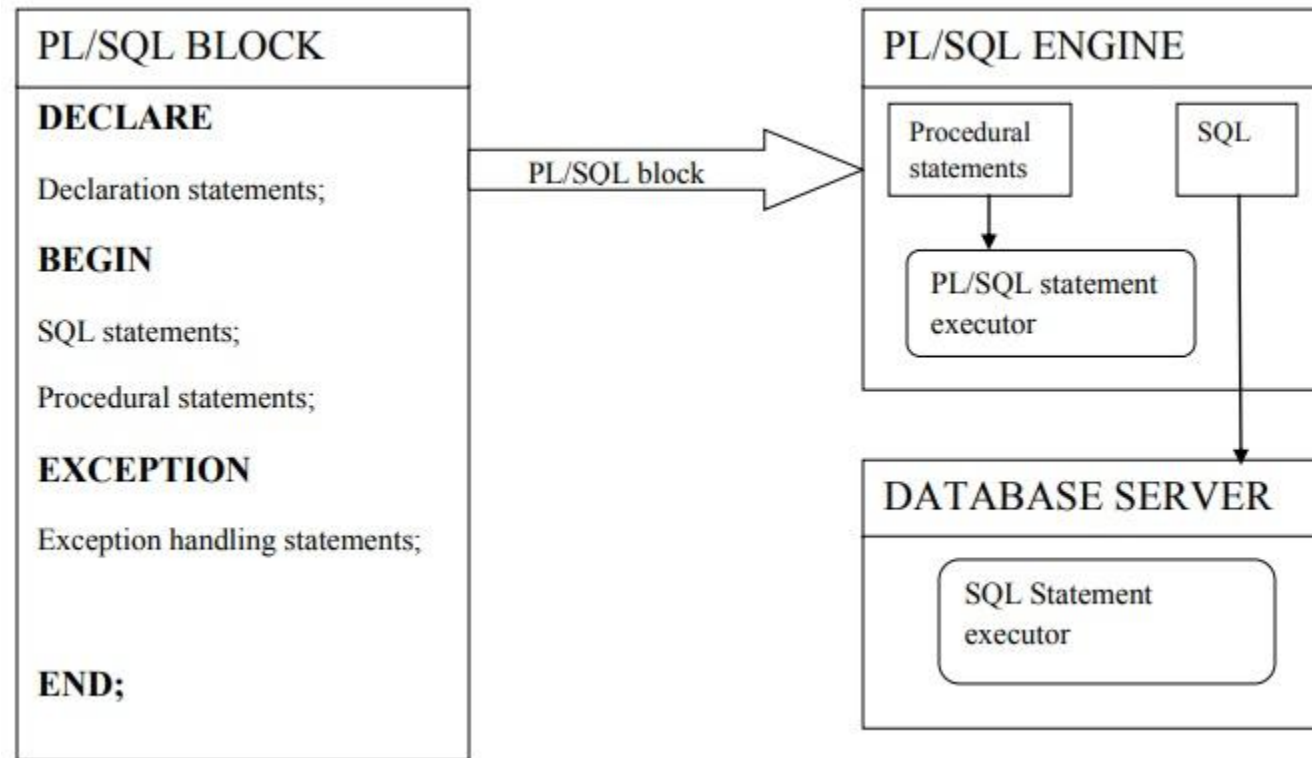
Introducció

- Extensió **procedural** de **SQL**:
 - Blocs
 - Loops, condicions
 - Funcions, procediments
 - Control d'errors.
 - Variables
 - ...
- Desenvolupat per Oracle.
- Síntaxi basada en Ada.

SQL vs PL/SQL

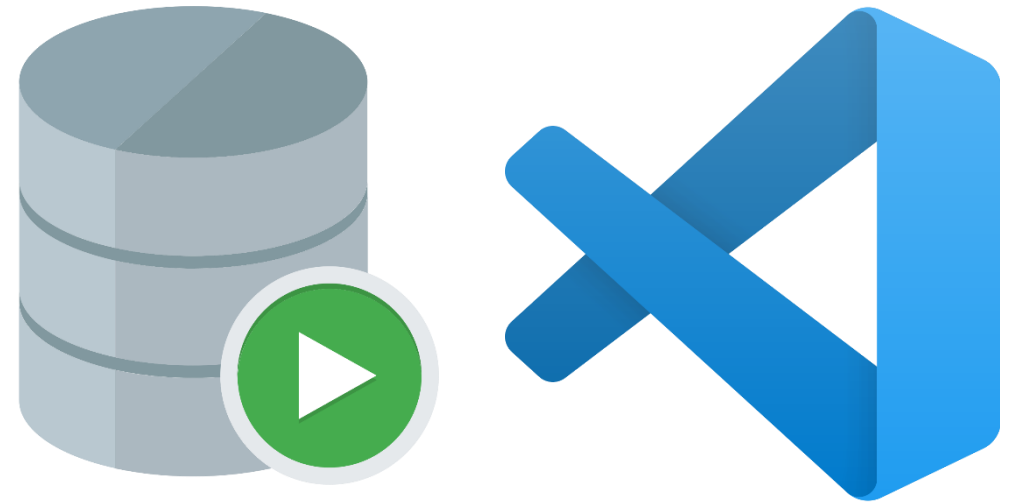
Basis of Comparison	SQL	PL/SQL
Definició	Structured Query Language (Llenguatge de consultes)	Llenguatge de programació de bases de dades utilitzant SQL.
Variables	No les soporta.	Soportades.
Estructures de control (for, if, ...)	No soportades.	Soportades
Orientació	Llenguatge orientat a les dades	Llenguatge orientat a les aplicacions.
Operacions	Consulta a consulta.	Les operacions s'executen en blocs (grups d'operacions), el que redueix l'ús de la xarxa.
Tipus de llenguatge	Declaratiu	Procedural
Embed	SQL pot utilitzar-se dins PL/SQL	PL/SQL no es pot utilitzar a SQL
Interacció amb el servidor	Interactua directament amb el servidor de base de dades.	No interactua directament amb el servidor de base de dades.
Control d'errors/excepcions	No	Sí
Serveix per	DDL, DML (inclou consultes).	Blocs de codi, funcions, procediments, disparadors, paquets.
Velocitat de processament	Baixa per moltes dades.	Alta per moltes dades.
Ús	Consultar, alterar, afegir, eliminar i manipular dades.	Desenvolupar aplicacions que mostrin informació de SQL d'una manera lògica.

Arquitectura PL/SQL



Eines de programació

- TOAD 
- SQL Developer
- Visual Studio Code (Oracle Developer Tools)
- ...



Documentació

PL/SQL:

- [Oracle Database Database PL/SQL Language Reference, 21c](#)
- [Oracle Database PL/SQL Packages and Types Reference, 21c](#)

SQL:

- [Oracle Database SQL Language Quick Reference, 21c](#)
- [Oracle Database SQL Language Reference, 21c](#)

Codi d'exemple

```
DECLARE
    qty_on_hand  NUMBER(5);
BEGIN
    SELECT quantity INTO qty_on_hand FROM inventory
        WHERE product = 'TENNIS RACKET'
        FOR UPDATE OF quantity;
    IF qty_on_hand > 0 THEN -- check quantity
        UPDATE inventory SET quantity = quantity - 1
            WHERE product = 'TENNIS RACKET';
        INSERT INTO purchase_record
            VALUES ('Tennis racket purchased', SYSDATE);
    ELSE
        INSERT INTO purchase_record
            VALUES ('Out of tennis rackets', SYSDATE);
    END IF;
    COMMIT;
END;
```

Unitats lèxiques

- Components individuals més petits del codi PL/SQL:
 - Agrupacions de caràcters
- 5 unitats lèxiques:
 - Delimitadors
 - Identificadors
 - Literals
 - Comentaris
 - Pragmas

Delimitadors

- “character, or character combination, that has a special meaning in PL/SQL”.

+

Operador d'addició

:=

Operador d'assignació

'

Delimitador de string

||

Operador de concatenació

/

Operador de divisió

=

Operador relacional (igual)

(

Delimitador d'expressió
o llista (inici)

)

Delimitador
d'expressió o llista (fi)

--

Comentari d'una línia.

- Llista completa: [PL/SQL Delimiters](#)

Identificadors

- Noms donats a un objecte PL/SQL
 - Constants, variables, excepcions, procediments, cursors, tipus, paquets...
 - Case-insensitive
 - Mida màxima = 30



- Exemples: registryDate, codiPersona, id, nomidi1

Paraules reservades i paraules clau

- Identificadors amb un significat especial que Oracle ja ha “reservat”.
 - Reserved words (paraules reservades): No es poden utilitzar.
 - Keywords (paraules clau): Es poden utilitzar en certes situacions.
- Exemples: BEGIN, SELECT, ELSE, END, VALUES
- Llista completa: [PL/SQL Reserved Words and Keywords \(oracle.com\)](https://www.oracle.com/plsql/topic-refs/reserved-words-keywords.html)

Paraules clau vs paraules reservades.

```
DECLARE
    CONSTANT number;
BEGIN
    CONSTANT := 3;
    DBMS_OUTPUT.PUT_LINE (CONSTANT);
END;
```

Paraula clau: CONSTANT

```
DECLARE
    BEGIN number;
BEGIN
    BEGIN := 3;
    DBMS_OUTPUT.PUT_LINE (BEGIN);
END;
```

Paraula reservada: BEGIN

Literals

- Valors explícits
 - “Values not represented by identifiers or calculated from other values”.
- PL/SQL inclou tots els literals SQL amb l’addició dels literals BOOLEAN

TRUE

FALSE

NULL

- Exemples (SQL): ‘Character literal’, 123, DATE ‘2023-01-01’;

Comentaris

- Una línia

`-- This is a single-line comment`

- Multi-línia

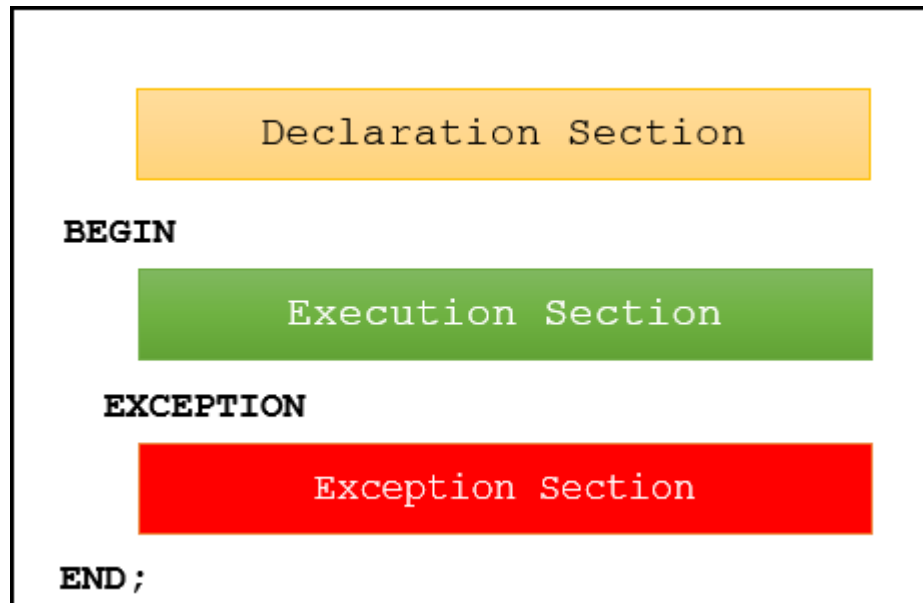
```
/*  
  This is a multi-  
  line comment  
*/
```

Pragmas

- Instruccions per el compilador SQL
- Exemples:
 - Possibilitat de cobertura de codi (tests).
 - Codi deprecats
 - ...
- Mirau: [PL/SQL Language Fundamentals \(oracle.com\)](https://www.oracle.com/plsql/language-fundamentals/)

Bloc anònim (anonymous block)

- Bloc: unitat bàsica d'un programa PL/SQL.
- Blocs anònims -> Blocks sense nom
 - No es guarden a la base de dades. D'un sol ús.



```
DECLARE --Opcional
    salutacio varchar2(12) := 'Hello World!';
BEGIN --Obligatori
    DBMS_OUTPUT.put_line (salutacio);
EXCEPTION --Opcional
    WHEN OTHERS THEN
        DBMS_OUTPUT.put_line ('Bye Bye');
END;
```


Variables

- Declaració amb nom, tipus i valor inicial(opcional)

`nom_variable tipus [NOT NULL] [:= valorInicial]`

- Ex: `preu number(10,2) := 19.4;`
- El valor inicial per defecte és NULL

- Assignació amb l'operador ":="

`v_lineNumber := v_lineNumber + 1;`

Destí

Expressió

Abast d'una variable

- PL/SQL permet l'ús de blocs niats (*nested*). Els blocs interns (*inner*) poden accedir a les variables dels seus blocs externs (*outer*).
- Tipus d'abast:
 - Variables locals: Declarades a un bloc intern, no accessibles per blocs externs.
 - Variables globals: Declarades al bloc més extern o a un paquet.
 - Accessibles des de qualsevol bloc intern.

Abast de variables: exemples.

```
DECLARE
    g_global1 varchar2(5) := 'Hello';
BEGIN
    DBMS_OUTPUT.PUT_LINE(g_global1);
    DBMS_OUTPUT.PUT_LINE(l_local1);

    DECLARE
        l_local1 varchar2(3) := 'Bye';
        g_global1 varchar2(4) := 'Hola';
    BEGIN
        DBMS_OUTPUT.PUT_LINE(l_local1);
        DBMS_OUTPUT.PUT_LINE(g_global1);
    END;
END;
```

Assignació de variables

- Examples:

```
in_stock := quantity > 0;
```

```
name := 'Cave Johson';
```

```
town := decode(cp, '07300', 'Inca', 'Unknown');
```

```
father := NULL;
```

Constants

- El seu valor no pot canviar després de la declaració. Inalterables.
- Declaració:

```
PI CONSTANT REAL := 3.14;
```

 - Ús de la paraula clau **CONSTANT**.
 - El valor inicial és **OBLIGATORI**.

SELECT INTO

- Assigna el resultat d'un SELECT a una variable o a un grup de variables.

- Exemples: `SELECT name INTO l_name FROM employees WHERE id = 2232;`

```
SELECT name, adress INTO l_name, l_adress FROM employees WHERE id = 2232;
```

Exemple



DECLARE

```
vNomEditorial VARCHAR2(10) := 'Planeta';  
vNomLlibre VARCHAR2(34) := 'Luna de Plutón';  
vIDEditorial NUMBER;
```

BEGIN

```
SELECT ID INTO vIDEditorial FROM EDITORIAL WHERE NOM = vNomEditorial;  
INSERT INTO LLIBRE(titol, ID_EDITORIAL) VALUES (vNomLlibre, vIDEditorial);
```

END;

Exercici

Sobre “HR.sql”:

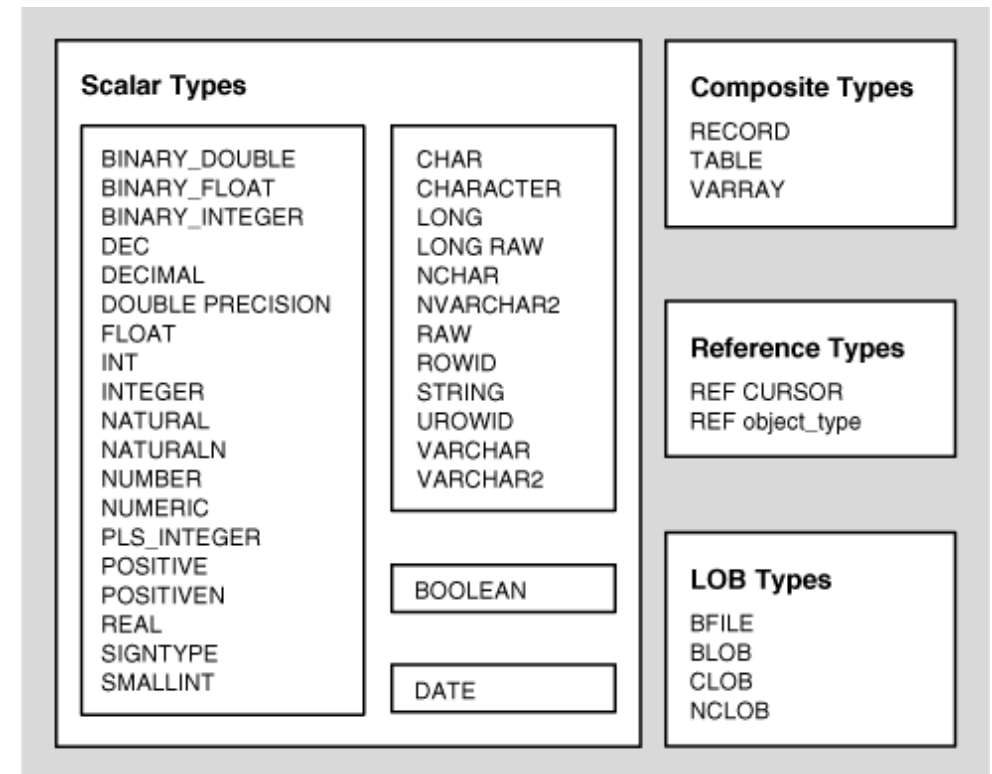
- A la taula “Countries” no hi ha tots els països: falta Espanya!



- Insereix el registre “Spain”:
 - Només pots inserir valors guardats a variables (3, una per cada columna de la taula).
 - Has d’obtenir la region ID corresponent de la taula “Regions” (per la regió de nom “Europe”) i guardar-la la variable corresponent.
 - Fes un INSERT INTO a COUNTRIES utilitzant les variables com a valors a inserir.
- Mostra els valors de les variables utilitzant DBMS_OUTPUT.PUT_LINE.

Tipus de dades

- Escalars: Sense components interns
 - Tipus de dades SQL
 - Veure [Data Types with Different Maximum Sizes in PL/SQL and SQL i PL/SQL Predefined Data Types](#)
 - BOOLEAN
 - PLS_INTEGER, BINARY_INTEGER
- Compostos: amb components interns
 - Col·leccions
 - Records



Nous tipus de dades escalars

- **BOOLEAN**
 - Valors booleans (TRUE, FALSE, NULL)
- **PLS_INTEGER i BINARY_INTEGER**
 - Són idèntics
 - Contenen sencers amb signe al rang [-2,147,483,648, 2,147,483,647] (32 bits)
 - Permet fer operacions més ràpides i requereix menys emmagatzematge que NUMBER i els seus subtipus.

RECORDA! No es poden utilitzar a sentències SQL

Conversions de tipus de dades

- Conversions implícites:
 - Conversions automàtiques fetes per PL/SQL d'un tipus a un altre.
 - Ex:

```
DECLARE
    v_quantity INTEGER := 1;
BEGIN
    DBMS_OUTPUT.PUT_LINE(v_quantity);
END;
```
- Conversions explícites: funcions per convertir entre tipus
 - TO_DATE, TO_TIMESTAMP
 - CAST
 - TO_NUMBER
 - TO_CHAR

Conversions String-Data: exemples

```
TO_DATE('10/03/2023', 'dd/mm/yyyy')
```

```
TO_CHAR(SYSDATE, 'mm/dd/yyyy')
```

```
TO_TIMESTAMP('10/03/2023 10:34:22', 'dd/mm/yyyy hh:mi:ss')
```

```
TO_CHAR(SYSTIMESTAMP, 'mm/dd/yyyy hh:mi:ss')
```

Conversions entre tipus: CAST FUNCTION

- Ús:

```
CAST(expressió AS tipus_desti)
```

- Exemples:

```
v_number number := CAST('4' AS NUMBER);  
v_char varchar2(1) := CAST(v_number AS VARCHAR2);  
v_char2 varchar2(20) := CAST(SYSDATE AS VARCHAR2);
```

Tipus de dades compostes

- Col·leccions:

- Components interns d'un mateix tipus de dades (ELEMENTS)
- Es pot accedir a cada element utilitzant un índex:
`c_employees(3);`
- Tipus: associative arrays, varrays, nested tables

- Records:

- Components interns de diferents tipus (FIELDS)
- Es pot accedir a cada camp pel seu nom:

`r_adress.city;`

Associative arrays

- Conjunt de parelles clau-valor
 - Clau: s'empra per trobar el valor associat. Índex únic.
 - Tipus de dades possibles per l'índex: VARCHAR2, PLS_INTEGER.
 - Valor: Valor del tipus de dades definit.
- Declaració del tipus i de la variable:

```
TYPE associative_array_type IS TABLE OF datatype [NOT NULL] INDEX BY index_type;  
associative_array associative_array_type;
```

- Ús (assignació i lectura): `associative_array(index)`

Associative arrays: exemple



```
DECLARE
    TYPE t_AA IS TABLE OF NUMBER INDEX BY VARCHAR2(20);
    v_marks t_AA;
BEGIN
    v_marks('Joan') := 6;
    v_marks('Sara') := 8;
    v_marks('Alberto') := 3;

    DBMS_OUTPUT.PUT_LINE('Average mark: ' || (v_marks('Joan') +
        v_marks('Sara') + v_marks('Alberto')) / v_marks.COUNT);
END;
```


Associative arrays: exercici

- Declara un associative array "sales" que contengui nombres (*number*) indexats per varchar2.
- Emplena 3 índexos 'Blake', 'Daisy', 'Lily' amb les vendes completades (status "Shipped") per els empleats (pista: SALESMAN_ID de ORDERS) amb aquests noms.
- Mostra el primer, el segon i el darrer element de la col·lecció (índex i valor).

Varrays (Variable-Size Arrays)

- Array amb un nombre d'elements variable des de zero (buit) a la mida màxima declarada.

- Declaració de tipus:

```
TYPE type_name IS VARRAY(max_elements) OF element_type [NOT NULL];
```

- Declaració de la variable i inicialització (necessari per fer-lo servir):

```
varray_name type_name [:= type_name(...)];
```

Varrays

- EXTEND: afegir un nou element (si nova mida > mida màx -> ERROR).

```
varray_name.EXTEND;  
varray_name(varray_name.last) := [...]
```

- DELETE: elimina tots els elements.

```
varray_name.DELETE;
```

- TRIM[(n)]: Elimina un o N elements del final del varray.

```
varray_name.TRIM
```

```
varray_name.TRIM(n)
```

Varrays: exemple

DECLARE

```
TYPE t_system is varray(5000) of varchar2(20);  
c_solarSystem t_system;
```

BEGIN

```
c_solarSystem := t_system('Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter',  
                           'Saturn', 'Uranus', 'Neptune');
```

```
c_solarSystem.EXTEND;
```

```
c_solarSystem(c_solarSystem.LAST) := 'Pluto';
```

```
DBMS_OUTPUT.PUT_LINE(c_solarSystem(c_solarSystem.LAST)|| ' torna ser un planeta');
```

```
c_solarSystem.TRIM;
```

```
DBMS_OUTPUT.PUT_LINE('El darrer planeta del Sistema Solar torna ser '  
                      ||c_solarSystem(c_solarSystem.LAST)||'. Pobre Pluto :(');
```

END;

BULK COLLECT

SELECT BULK COLLECT INTO permet recuperar i emmagatzemar al varray múltiples elements (més d'un valor de columna) des de la BBDD.

```
SELECT column_name bulk collect into varray_name from ...
```

Exemple:



```
DECLARE
    TYPE t_varr IS VARRAY(3) OF LLIBRE.TITOL%TYPE;
    v_arr t_varr;
BEGIN
    SELECT titol BULK COLLECT INTO v_arr FROM LLIBRE ORDER BY exemplars DESC FETCH FIRST 3 ROWS ONLY;
    DBMS_OUTPUT.PUT_LINE('El llibre amb més exemplars és '||v_varr(1));
END;
```

Varrays: exercici

Sobre “HR.sql”

1. Guarda els noms dels 3 empleats amb més comandes (*order shipped*) a un Varray ordenant-los de més a menys.
2. Mostra el nom dels empleats (DBMS_OUTPUT).
3. Elimina (TRIM) el 2n i 3r empleats amb més comandes *shipped* i comprova que el nombre final d'elements al Varray és 1. Mostra el nom de l'empleat que queda al Varray.

Nested tables

- Guarda un nombre de files indeterminat.
- Declaració del tipus:

```
TYPE nested_table_type IS TABLE OF element_datatype [NOT NULL];
```

- Declaració de la variable i inicialització:

```
nested_table_variable nested_table_type [:= nested_table_type(...)];
```

Nested tables

- Afegir elements:

```
nested_table_variable.EXTEND;  
nested_table_variable := element;
```

- Accedir a elements:

```
nested_table_variable(index);
```


Nested Tables vs Varrays

Varray	Nested Table
Nombre d'elements declarat (Mida fixa). No pot augmentar més enllà del seu límit.	Mida ampliable de manera dinàmica.
Dens.	Inicialment dens, però pot convertir-se en dispers.

Array of Integers

321	17	99	407	83	622	105	19	67	278
x(1)	x(2)	x(3)	x(4)	x(5)	x(6)	x(7)	x(8)	x(9)	x(10)

Fixed
Upper
Bound

Nested Table after Deletions

321		99	407		622	105	19		278
x(1)		x(3)	x(4)		x(6)	x(7)	x(8)		x(10)

Upper limit
of index
type

Usos apropiats

Associative arrays	Varrays	Nested tables
Taula de consulta relativament petita, que pot ser construïda en memòria cada vegada que s'invoca el subprograma o s'inicialitza el paquet que el declara. Pas de col·leccions a i des de el servidor de base de dades.	Es coneix el nombre màxim d'elements. S'accedeix als elements seqüencialment.	Nombre d'elements indeterminats. Els valors dels índexs no són consecutius. No s'han d'eliminar o actualitzar tots o molts elements simultàniament, només alguns.

Equivalència dels tipus compostos

Non-PL/SQL Composite Type	Equivalent PL/SQL Composite Type
Hash table	Associative array
Unordered table	Associative array
Set	Nested table
Bag	Nested table
Array	VARRAY

Mètodes de les col·leccions

Method	Type	Description
DELETE	Procedure	Elimina els elements d'una col·lecció.
TRIM	Procedure	Elimina elements del final d'un varray o nested table.
EXTEND	Procedure	Afegeix elements al final d'un varray o nested table.
EXISTS	Function	Retorna TRUE si l'índex especificat existeix a la col·lecció.
FIRST	Function	Retorna el primer índex de la col·lecció.
LAST	Function	Retorna el darrer índex de la col·lecció.
COUNT	Function	Retorna el nombre d'elements a la col·lecció.
LIMIT	Function	Retorna el nombre màxim d'elements que pot tenir una col·lecció.
PRIOR	Function	Retorna l'índex anterior a l'índex especificat.
NEXT	Function	Retorna l'índex posterior a l'índex especificat.

Tipus de col·lecció globals

- Es poden declarar tipus de dades compostos i guardar-los a la base de dades perquè es puguin emprar fora d'un bloc PL/SQL específic.

- Varray:

```
CREATE [OR REPLACE ] TYPE type_name AS | IS  
    VARRAY(max_elements) OF element_type [NOT NULL];
```

- Nested table:

```
CREATE [OR REPLACE] TYPE nested_table_type  
    IS TABLE OF element_datatype [NOT NULL];
```

- Els associative arrays han d'estar a un paquet/procediment.

Comparació de tipus de col·lecció

Collection Type	Number of Elements	Index Type	Dense or Sparse	Uninitialized Status	Where Defined	Can Be ADT Attribute Data Type
Associative array (or index-by table)	Unspecified	String or PLS_INTEGER	Either	Empty	In PL/SQL block or package	No
VARRAY (variable-size array)	Specified	Integer	Always dense	Null	In PL/SQL block or package or at schema level	Only if defined at schema level
Nested table	Unspecified	Integer	Starts dense, can become sparse	Null	In PL/SQL block or package or at schema level	Only if defined at schema level

Records

- Declaració de tipus:

```
TYPE record_type IS RECORD (  
    field_name1 data_type1 [[NOT NULL] := | DEFAULT default_value],  
    field_name2 data_type2 [[NOT NULL] := | DEFAULT default_value],  
    ...  
);
```

- Declaració de la variable:

```
record_name record_type;
```

- Es pot utilitzar %ROWTYPE per copiar els tipus de les columnes d'una taula, sense necessitat d'especificar cada camp ni de declarar un tipus.

Records

- Accedir als camps (llegir/assignar valor):

```
record_name.field_name
```

- Els records es poden assignar a altres records del mateix tipus, però no es poden comprar (s'han de comparar els camps, un per un).
- SELECT INTO i UPDATE es poden utilitzar amb tot un record:

```
SELECT * INTO record_variable_name;
```

```
UPDATE table_name SET ROW = record_variable_name WHERE condition;
```


Records: example

```
DECLARE
  TYPE r_employee IS RECORD (
    employee_id EMPLOYEES.EMPLOYEE_ID%TYPE,
    first_name EMPLOYEES.FIRST_NAME%TYPE,
    last_name EMPLOYEES.LAST_NAME%TYPE,
    email EMPLOYEES.EMAIL%TYPE,
    phone EMPLOYEES.PHONE%TYPE,
    hire_date EMPLOYEES.HIRE_DATE%TYPE,
    manager_id EMPLOYEES.MANAGER_ID%TYPE,
    job_title EMPLOYEES.JOB_TITLE%TYPE
  );
  v_employee r_employee;
BEGIN
  SELECT * INTO v_employee FROM (SELECT * FROM EMPLOYEES ORDER BY HIRE_DATE DESC) WHERE ROWNUM=1;
  DBMS_OUTPUT.PUT_LINE(v_employee.first_name);
  v_employee.first_name := 'Pere';
  v_employee.employee_id := "OT"."ISEQ$$_73411".nextval;
  v_employee.hire_date := SYSDATE;
  INSERT INTO EMPLOYEES VALUES v_employee;
END;
```

Records: exercici

Sobre “HR.sql”

- G.Skill ha llançat un nou kit de memòria com a part de la seva línia de productes “G.Skill Ripways V Series” amb una velocitat millorada (DDR4-3600) i preu (Standard cost: 697.56, list price: 779.99).
- Utilitza un record per obtenir dades d’un producte “G.Skill Ripjaws V Series” existent i altera el seu preu i la seva descripció, indicant la nova velocitat. Insereix aquestes noves dades a la taula PRODUCTS com un nou registre.

Records amb col·leccions

Es poden utilitzar col·leccions de records per guardar múltiples elements d'una estructura de dades pròpia o elements copiats dels registres d'una taula (%ROWTYPE).

S'ha de:

1. Declarar el tipus del record (si és propi).
2. Declarar el tipus de la col·lecció amb el tipus del record com el tipus dels valors (o amb TAULA%ROWTYPE si copiam l'estructura dels registres d'una taula).

Records amb col·leccions: exemple



```
DECLARE
  TYPE t_varr IS VARRAY(3) OF EMPLOYEES%ROWTYPE;
  l_varr t_varr;
BEGIN
  SELECT * BULK COLLECT INTO l_varr FROM EMPLOYEES ORDER BY SALARY DESC FETCH FIRST 3 ROWS
  ONLY;
  DBMS_OUTPUT.PUT_LINE('Els 3 empleats que cobren més són:');
  FOR i IN l_varr.FIRST..l_varr.LAST LOOP
    DBMS_OUTPUT.PUT_LINE(l_varr(i).first_name || ' ' || l_varr(i).last_name);
  END LOOP;
END;
```

Records amb col·leccions: exercici

Sobre “U6_Biblioteca.sql”:

1. Guarda els 3 llibres amb més exemplars dins un VARRAY de records de LLIBRE.
2. Imprimeix en pantalla (DBMS_OUTPUT.PUT_LINE) el títol dels 3 llibres i el seu autor, indicant l'ordre.